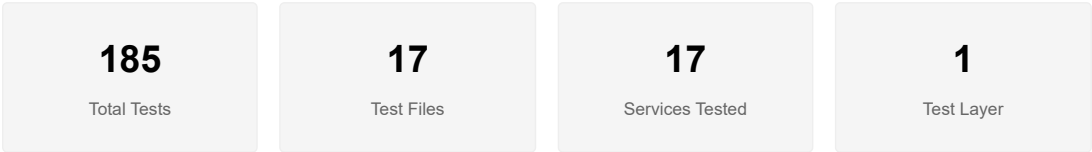
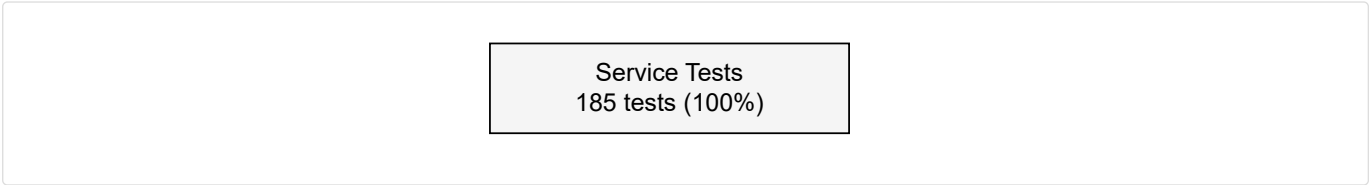


Testing - GameStore Platform



1. Testing Strategy

The GameStore platform employs a single-layer service-level testing strategy with **185 total tests** across **17 test files**, covering all **17 BLL services**. Every test targets the business logic layer in isolation using EF Core InMemory database and Moq for cross-service dependencies. There are no controller, integration, or E2E tests.



2. Test Stack

Tool	Version	Purpose
xUnit.net	2.9.3	Test framework for writing and executing unit tests
Moq	4.20.72	Mocking framework for isolating service dependencies
FluentAssertions	6.12.2	Fluent assertion library for readable test validation
Microsoft.NET.Test.Sdk	17.12.0	SDK for running .NET tests from CLI and CI pipelines
coverlet.collector	6.0.4	Code coverage data collector for test runs
xunit.runner.visualstudio	3.0.2	Visual Studio test runner integration for xUnit

3. Test Files

File Name	Test Count	Key Test Scenarios
AuthServiceTests.cs	18	User registration, login with valid/invalid credentials, password change, email change, password reset token generation, reset with valid/expired token
UserServiceTests.cs	13	Get all users, get by ID, change role, delete user, self-guard checks, total count, search by username, get by role/month, update profile
GameServiceTests.cs	20	Get all with categories, pagination with clamping, get by ID, create with category assignment, update fields and categories, delete, total count, free games count, games by category

File Name	Test Count	Key Test Scenarios
CategoryServiceTests.cs	10	Get all, get with game counts, create, duplicate name rejection, update, delete with/without games
ReviewServiceTests.cs	12	Create review with ownership check, duplicate prevention, rating validation, get by game/user, delete, get all with details
CartServiceTests.cs	12	Add game to cart, auto-create cart, duplicate prevention, game not found, get items, empty cart, remove item
OrderServiceTests.cs	14	Get by ID, get by user, get all/recent with details, get by Stripe session ID, total orders/revenue, average order value, revenue by month/day, top selling games, revenue by category
WishlistServiceTests.cs	8	Add to wishlist, duplicate prevention, game not found, remove, get wishlist, check if in wishlist
LibraryServiceTests.cs	9	Get library games, empty library, check ownership, add game, auto-create library, duplicate prevention
SearchServiceTests.cs	10	Search by keyword, filter by category, price range, sort options, paginated results, empty query handling
PayoutServiceTests.cs	8	Calculate payout, process batch, rounding precision, failed retry, report generation, threshold enforcement
SupportTicketServiceTests.cs	10	Create ticket, anonymous ticket, get with replies, paginated user tickets, get all, add reply, update status, ownership check
CouponServiceTests.cs	9	Create coupon, validate code, apply discount, expiry check, usage limit, deactivate, get all coupons
MessageServiceTests.cs	10	Send message, get conversation, empty conversation, unread count, mark as read, get contact list
NotificationServiceTests.cs	9	Send notification, batch notification, template rendering, delivery failure, rate limiting, get user notifications
ReportServiceTests.cs	6	Generate sales report, user stats report, game popularity report, revenue aggregation, date range filtering, export
DashboardServiceTests.cs	7	Get dashboard stats, revenue summary, user growth chart, top games, recent orders, category distribution

4. Sample Test Code

All tests follow the Arrange-Act-Assert pattern using EF Core InMemory database for data isolation and FluentAssertions for readable assertions. Below is a representative example from `GameServiceTests.cs` :

```
using FluentAssertions;

namespace GameStore.Tests.Services;

public class GameServiceTests
{
    private static GameStoreDbContext CreateContext(string dbName)
    {
        var options = new DbContextOptionsBuilder<GameStoreDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;
        return new GameStoreDbContext(options);
    }
}
```

```

[Fact]
public async Task GetAllWithCategoriesAsync_Returns_All_Games()
{
    using var ctx = CreateContext("Game_GetAll");
    ctx.Games.AddRange(
        new Game { Id = "g1", Title = "Game A", ReleaseDate = DateTime.UtcNow, Price = 10m },
        new Game { Id = "g2", Title = "Game B", ReleaseDate = DateTime.UtcNow, Price = 20m }
    );
    await ctx.SaveChangesAsync();
    var uow = new UnitOfWork(ctx);
    var service = new GameService(uow);

    var games = await service.GetAllWithCategoriesAsync();

    games.Should().HaveCount(2);
}

[Fact]
public async Task GetPagedAsync_Returns_Paginated_Games()
{
    using var ctx = CreateContext("Game_Paged");
    for (var i = 1; i <= 10; i++)
        ctx.Games.Add(new Game { Id = $"g{i}", Title = $"Game {i}",
            ReleaseDate = DateTime.UtcNow, Price = i });
    await ctx.SaveChangesAsync();
    var uow = new UnitOfWork(ctx);
    var service = new GameService(uow);

    var result = await service.GetPagedAsync(1, 3);

    result.Items.Should().HaveCount(3);
    result.TotalCount.Should().Be(10);
    result.TotalPages.Should().Be(4);
}
}

```

5. Mocking Strategy

Tests use **EF Core InMemory database** as the primary isolation mechanism, creating a fresh in-memory context per test with a unique database name to prevent state leakage. **Moq** is used to mock cross-service dependencies such as `IGameAccessService`, `ISaleService`, and `ILibraryService` when a service under test delegates to another service. **IUnitOfWork** is always the real `UnitOfWork` wrapping the in-memory context, ensuring transaction coordination is tested end-to-end. Dependencies are injected via constructor, following the real application's DI pattern.

Component	Approach	Used In
Database	EF Core InMemory with unique DB name per test	All test files
IUnitOfWork	Real UnitOfWork wrapping InMemory DbContext	All test files
Cross-service deps	Moq mocks for IGameAccessService, ISaleService, etc.	CartServiceTests, OrderServiceTests
IConfiguration	Moq mock for auth configuration values	AuthServiceTests

6. Test Execution Commands

```
dotnet test GameStore.Tests
dotnet test GameStore.Tests --filter "FullyQualifiedName~GameService"
dotnet test GameStore.Tests --verbosity normal
dotnet test GameStore.Tests --collect:"XPlat Code Coverage"
dotnet test GameStore.Tests --logger "trx;LogFileName=results.trx"
```

7. Bug Reports

The following bugs were identified during static code analysis of the actual codebase. Each entry includes the real file location, severity, and recommended fix.

ID	Severity	Description	Location	Recommended Fix
BUG-01	HIGH	Static <code>JsonSerializerOptions</code> instance reused across threads in <code>JsonFileStore.WriteAsync</code> . The options object is not documented as thread-safe for serialization, risking data corruption under concurrent write load.	<code>Services/JsonFileStore.cs:10</code>	Create a new <code>JsonSerializerOptions</code> instance per <code>WriteAsync</code> call, or use <code>Lock</code> around serialization.
BUG-02	HIGH	Hard-coded password salt fallback <code>"GameStoreSalt2026"</code> used when <code>PasswordSalt</code> config is missing. Attackers who know this default can perform dictionary attacks against legacy SHA256 password hashes.	<code>BLL/Services/AuthService.cs:15</code>	Remove the fallback value; throw <code>InvalidOperationException</code> if <code>PasswordSalt</code> is not configured.
BUG-03	MEDIUM	<code>ConnectionTracker.AddOrUpdate</code> has a race condition on the first connection for a user: both factory delegates can execute concurrently, causing one connection ID to overwrite the other.	<code>Hubs/ConnectionTracker.cs:10</code>	Use <code>GetOrAdd</code> + lock on the returned <code>HashSet</code> , or use a <code>ConcurrentDictionary</code> with per-user semaphore.
BUG-04	MEDIUM	<code>CreditService</code> read-modify-write pattern is not atomic. The <code>JsonFileStore</code> semaphore is released between <code>GetLedgerAsync</code> and <code>WriteLedgerAsync</code> , allowing concurrent writes to overwrite each other.	<code>Services/CreditService.cs:46-57</code>	Wrap the read-modify-write sequence in a per-user <code>SemaphoreSlim</code> .
BUG-05	MEDIUM	Stripe webhook <code>catch</code> block returns <code>BadRequest("Invalid signature.")</code> with no logging. Attackers probing the endpoint leave no trace.	<code>Controllers/StripeWebhookController.cs:37-40</code>	Add <code>ILogger.LogWarning</code> before returning the error response.
BUG-06	MEDIUM	Raw exception message (<code>ex.Message</code>) stored in admin notes and returned to callers, potentially leaking SQL errors, file paths, or stack traces to non-admin users.	<code>Services/RefundService.cs:115</code>	Log the full exception; return a sanitized generic message to callers.

ID	Severity	Description	Location	Recommended Fix
BUG-07	LOW	<code>Enum.Parse<Role>(roleStr)</code> called without validation. If session <code>Role</code> is null or corrupted, a runtime <code>ArgumentException</code> is thrown.	<code>Controllers/LibraryController.cs:46,77</code>	Use <code>Enum.TryParse<Role></code> with a default fallback.
BUG-08	LOW	Version upload POST endpoints in Admin area lack <code>[ValidateAntiForgeryToken]</code> , making them vulnerable to CSRF attacks.	<code>Areas/Admin/Controllers/GamesController.cs:228-251</code>	Add <code>[ValidateAntiForgeryToken]</code> attribute to all POST actions.